Testing

How to run a test

Steps to run the test:

1. Download all dependencies
2. Open your virtual environment to use pytest
3. Call the test, from the “tests” folder so you can use the fixtures on conftest.py
    a. pytest ./{insert_folder}/{insert filename, OPTIONAL}
    b. pytest ./{insert_folder}/{insert filename, OPTIONAL} –browser {Default/Safari/Edge/Chrome/Firefox}
    c. pytest ./{insert_folder}/{insert filename, OPTIONAL} {Browser Fixture} {added arguments}

See the pytest documentation on the read me for pytest specific information

Why the tests are structured like that:

Each test file inside the “tests” folder follows the naming convention of having test in the front of the file name and in any functions that are tests to be run. This is done so that pytest can discover all tests throughout the given folder without need for a driver.

Each component being tested, if different enough, has its own folder. For example, the react components have their own folder with their component tests inside, but there is also a test for the browser fixture, which has its own folder as well.

Inside each test is a preamble that contains the author, their role, and the tests that are contained within the file. Please note it is likely there are more functions than tests within any given folder.

Each of the e2e tests uses selenium to function, and a fixture to run across all browsers. The fixture “browser_types_fixture” handles the creation process for the webdriver for a given browser. This fixture used the binaries folder to identify the location of the browsers Firefox and Chrome. You can copy their executables here for the tests to function, otherwise it will use your systems path to identify the location of the browsers. (Edge and Safari are found using the system paths).

The tests also use a e2e_helper_script that creates functions often used across tests, please see that file for more details.

Finally, e2e tests also import a variable for the browser superlist, which is used to create the pytest parameterizations.

How to add more tests:
You can add as many tests as you want. To do so create a new folder in the tests folder, create your tests in following with pytest conventions, and add your code. You can look at other tests for how to use the fixtures, import functions from the e2e_script_helpers, and import variables.

Areas for improvement:
There are more mini-functions that could be added to the e2e_script_helpers file, and inside components. Overall there are no known MAJOR issues with this current set up.